Unit 6 Organizational Metrics

1) Metric Taxonomy

→ This concept is about how organizations structure and classify metrics when running experiments, especially A/B tests. Instead of treating all metrics equally, the book categorizes them into roles, just like people in a company have different responsibilities.

Metrics

1. Goal (a.k.a OEC, North Star, or Success Metric).

- What: The ultimate outcome you want to improve.
- Example: Revenue, long-term user retention, customer satisfaction
- Why it matters: All other metrics should support this.

> Analogy: Reaching your destination (e.g. Pokhara in a road trip). This is your final goal.

2. Driver Metrics (a.k.a. leading indicators) (Sign-post, surrogate)
   - What: Metrics that influence or predict the goal metric
   - Examples: Conversion rate, click-through rate, add-to-cart
   - Why it matters: Drivers give faster feedback and help teams iterate quickly

> Analogy: Your current speed. The faster you go (without crashing), the quicker you reach Pokhara.

3. Guardrail Metrics.

- what: Metrics that protect you from unintended harm while optimizing the goal.
- Example: Page load time, crash rate, unsubscribe rate, margin
- why: They set boundaries — don't let improvement in the goal cause other areas to fail.

> Analogy: Your car's engine temperature or tire pressure. If ignored, your journey fails — even if you're fast.

4. Diagnostic Metrics.

- what: Not the main focus, but help debug if something is off.
- Examples: Drop-off rate at specific funnel steps, latency at different endpoints
- why: Useful when goal or driver metrics behave unexpectedly

> Analogy: Like checking your GPS or route recalculating when lost. Not your main focus, but helpful.

2) Math Concept - How these connect.

Say we have

Revenue = Visitors × Conversion Rate × Average Value.

- Revenue = Goal
- Conversion Rate & AOV = Drivers
- If Page Load Time increases, it might Conversion Rate → lower Revenue.

So we measure cause-effect chains mathematically using multiplicative models or regression, and track changes using delta % uplift in A/B testing.

(3) Interlocks - How this connect with other units.

Concept	Link
OEC (Overall Eval. Criteria)	Goal metric design & prioritization (unit 7)
Speed Matters	Proves page speed is a driver or a guardrail (unit 5)
Gameability	Warns that drivers metrics can be exploited if not monitored.
Guardrail Metrics	Further detailed in SRM & A/A - like a firewall for experiments.
SRM & Experiment Validity	Guardrails also include data quality checks (later chapters).

(4) Summary: Build a Metrics Tree.

Think of your experiment like a tree:

- Root / Trunk = Goal metric (your product's core value)
- Branches = Driver metrics (levers you can pull)
- Fences = Guardrails (boundaries to protect system)
- Bark sensors = Driver metrics (tell you where bugs or stress point are)

Together, this taxonomy helps you decide:

- What to optimize.
- What to monitor.
- What to protect.

(2) Formulating Metrics: Principles & Techniques.

Below you'll find:

1. Core principles the book recommends when designing a metric.
2. Concrete technique for turning raw event logs into
3. A "Coffee-shop" analogy to keep things intuitive.
4. Math insight: showing why some formulations outperform others.
5. A short python demo (see output above) that contrasts two ways to calculate conversion rate.

1. Principle (the checklist).

#	Principle	Short test
P1	Alignment	"If this metric moves 0.↑, do we definitely get closer to the business goal?"
P2	Sensitivity	"Will typical A/B test uplifts (≈ 1–5%) register as statistically significant within days/weeks?"
P3	Directionality	"Is ↑ unambiguously good (or bad)?" Never mix good & bad events in one number.
P4	Unit Consistency	Use the same unit you randomised on (user, session, cookie)
P5	Variance control	Prefer formulations that lower noise without hiding signal (ratio metrics, CUPED, winsorisation).
P6	Resistance to gaming	Could a team artificially boost it without real user value? (link to Gameability sidebar)

P7	Explainability	APM should be able to recite the definition in one breath; otherwise people won't trust it.
P8	Timeliness	How soon after the experiment starts can it be evaluated? (early vs. late binding events).

⑧ Techniques (the tool-box)

Technique	What it does	Typical math.
Ratio metrics	Normalise volume differences (e.g. conversion / sessions)	$\hat{r} = \frac{\sum c_i}{\sum s_i}$
per-users averaging	Aligns with user-level randomisation, reduce skew	$\bar{r} = \frac{1}{N} \sum_i \frac{c_i}{s_i}$
CUPED	Uses a pre-period covariate to cut variance by 20–50%	$Y = Y - \theta(X - \bar{X})$
Winsorization/ capping	Trims extreme outliers (& heavy-tailed dev)	Clamp top 99.9%ile

log transform	Stabilises variance for revenue & latency	Analyse ln(1+R)
Event funnels	Break multi-step tasks into drop-off rates; each step can be a driver metric	Paid-> pay = payers / add-to-cart.
Time-window binding	Decide when an event "belongs" to an experiment (e.g. 14-day re-exposure)	Right-censoring / survival curves.

(3) Analogy: Running a coffee-shop chain.

Shop action	Raw events	Metric we need	Principle illustrated
People enter shop	entrance	Foot-fall rate = entrances ÷ open hours	Ratio metric.
Orders placed	orders	Conversion = orders ÷ entrances	Unit consistency.
Avg spend per order	revenue	AOV = revenue ÷ orders	Alignment
Barista error count	mistakes	Quality guardrail (must ↓)	Directionality
Net promoter score surveys	NPS response	NPS (driver for loyalty)	Explainability.

(2) Real-World Case: Food Delivery App: (workflow).

Goal: Test if a new upsell feature increases Average Order Value (AOV)

A/B Test Simulation summary

We split 5,000 users into control and treatment. Each user:

- Has varying number of sessions
- May or may not convert (place an order)
- Spends a different amount when they do convert.

Results

Group	AOV (Ruby)	Conversion Rate	Users.
Control	$24.72	19.56%	2504
Treatment	$27.03	21.6%	2496.

Statistical significance p-value = 0.0000
So the treatment effect is real.

Metric Formulation Principle Applied.

Principle (Px)	How it was applied.
P1: Alignment	Higher AOV directly increases revenue → aligned with business goal.

P2: Sensitivity — Test picked up 1.5% uplift in conversion and $2 increase in AOV.

P3: Directionality — AOV↑ = always good. No ambiguity.

P4: Unit Consistency — Per-user AOV used, since users were randomised.

P5: Variance Control — Used per-user averages instead of raw orders (less noise)

P6: Resistance to Gaming — User can't fake AOV: it reflects actual money spent

P7: Explainability — Metric = "Total Revenue ÷ Total Orders". Easy to explain.

P8: Timeliness — Evaluated in 1 week, early metric (timely feedback).

Math Techniques Used

Technique	Description	In Code
Ratio metric	Global AOV = total revenue ÷ total conversion	✓
Per-user averaging	AOV per users = avg (order value ÷ conversion)	✓

Variance Control	Used per-user means to reduce weight of power users.	✓
Significance test	Welch's t-test for per-user AOV	✓
Distribution plot	Histogram showing lift in treatment group	✓

③ Coffee-Stop Analogy – Full Math Workflow.

Step	What we did	Metric Principles shown
1. Simulate data	30 control vs 30 treatment stores, 14 days each	Unit consistency (store-day)
2. Formulate metrics	Goal: daily revenue Drivers: conversion rate & AOV Guardrail: avg wait.	Alignment, Directionality.
3. Cut variance	Used per-store means (not raw or days) to reduce noise	Variance control, Sensitivity.
4. Test significance	Welch's t on revenue, conversion, AOV, wait time	Sensitivity, Timeliness
5. Visualise	Histogram show clear right-shift in treatment revenue	Explainability.

Key Numbers

Metric (per store)	Control	Treatment	p-value
Daily Revenue	$236.6	$285.1	< 0.001
Conversion Rate	30.0%	32.0%	< 0.001
AOV	$3.99	$4.29	< 0.001
Avg wait (guardrail)	4.97 min	5.54 min	< 0.001.

- Goal metric improved by ~ $42/day.
- Driver metrics (conversion & AOV) both up → validates causal story.
- Guardrail wait time worsened +0.6 min, but still < 7 min limit.

Why this Hits Every Principle

1. Simple & stable: Revenue is $; customers & execs get it.
2. Aligned: More orders × higher price ⇒ more revenue.
3. Actionable: Teams change store layout to move those drivers.

4. Sensitive – 14 days detected 2 ppt conversion lift.

6. Explainable – Histogram + table shows the whole story at a glance.

7. Guardrail – wait-time protects customer experience.

Treat it a complete, end-to-end math example of metric formulation, variance handling, and statistical testing – all inside the coffee-shop analogy..

(3) Evaluating Metrics.

(1) What is Evaluating Metrics?

→ When you choose a metric (like conversion rate or minutes watched), you need to check if it's good metric.

(2) Four Question to Ask About Any Metric?

	Question	Quick Math check
Q1 Alignment	Does the metric move in the same direction as the long-term goal?	Correlation ($\rho \geq 1$) between metric lift and goal lift across past experiment.
Q2 Sensitivity (Power)	Does it detect small but real changes quickly?	Signal-to-Noise = Δ/σ; or % experiments where metric sign = goal sign.

Q3 Robustness	Is it stable across segments, days, data glitches?	A/A tests, variance inflation factor, missing-data audits.
Q4 Interpretability	Do stakeholders instantly know what ↑ means	one-sentence meaning test.

| Fail any one badly → rethink or replace the metric

③ Real-world Analogy: car dashboard.

Gauge	Good?	Evaluation check
Speedometer	✓	Aligned with "how fast I travel", responds instantly, easy to read.
wind-noise meter	⚠	Maybe correlate with speed, but noisy & segment-dependent.
odometer	✓	perfectly aligned with distance but not sensitive in 1-sec windows.

② Choosing the Right Thermometer

Imagine you're measuring fever (the real goal). You test two thermometers (Metric A and Metric B).

Feature	Thermometer A (Good)	Thermometer B (Bad)
Shows fever rise?	✓ Yes, clearly	⚠ Sometimes wrong.
Respond fast	✓ Yes (real-time	✗ Too noisy.

A = Aligned + sensitive → Trustworthy
B = Unreliable = Bad Metrics.

(4) Math Background.

→ Evaluating Metrics is about checking:
"Can this metric be trusted to reflect the real goal of our product?"

Just like checking if a thermometer gives an accurate reading for 2 days, we want to know if

- Is this metric aligned with our goal?
- Is it sensitive enough to detect real change?
- Is it noisy, biased or gameable?

The Core Equation (Mental Model).

$$\Delta M = \alpha \cdot \Delta G + \epsilon$$

where:
- ΔG : the true change in goal (e.g. revenue).
- ΔM : the metric is measured change (e.g. clicks)
- α : alignment strength.
- ϵ : noise (random junk).

This tells us:
- If $\alpha = 1$, your metric perfectly tracks the goal.
- If $\sigma^2 = Var(\epsilon)$ is small, your metric is stable.

1. Alignment

The Pearson correlation between ΔM and ΔG is

$$\rho = \frac{Cov(\Delta M, \Delta G)}{\sqrt{Var(\Delta M)\, Var(\Delta G)}} = \frac{\alpha T^2}{\sqrt{(\alpha^2 T^2 + \sigma^2) T^2}}$$

$$= \frac{\alpha T}{\sqrt{\alpha^2 T^2 + \sigma^2}} \quad \cdots (2)$$

where $T^2 = Var(\Delta G)$

Perfect alignment $\Rightarrow \alpha = 1$ and $\sigma^2 \to 0 \Rightarrow \rho \to 1$

~~2. Sensitivity (stabi~~

Meaning: Does it move in the same direction as the goal?

2. Sensitivity (statistical Power).

2.1 Variance of the metric estimator

For an A/B test with n users per variant

$$Var(\widehat{\Delta M}) = \frac{2(\alpha^2 T^2 + \sigma^2)}{n} \quad \cdots (3)$$

2.2 Detectable effect (MDE)

Smallest goal-lift Δ^* (at 2-sided level α_0, power $1-\beta$) that propagates through (1):

$$\Delta^* = \frac{z_{1-\alpha_0/2} + z_{1-\beta}}{\alpha} \sqrt{\frac{2(\alpha^2 T^2 + \sigma^2)}{n}}$$

where z_q is the q-quantile of $N(0,1)$.

Lower σ^2 or higher $\alpha \rightarrow$ smaller MDE $\rightarrow$ higher sensitivity.

Meaning: Can it detect small changes? (smaller is better)

3. Diagnostic Quantities.

Property	Formula	Interpretation
Scaled bias	$b = \Sigma \lvert \Delta M \rvert - \Delta G$ $= (\alpha - 1) \Sigma \lvert \Delta q \rvert$	should be ≈ 0 if $\alpha \approx 1$. is it under/over estimating the true effect.
Noise Ratio	$r = \frac{\sigma^2}{q^2 T^2}$	Extra variance vs. good signal. Is it mostly noise? (lower = better)
Relative efficiency	$E = \frac{Var\ \widehat{\Delta G}}{Var(\widehat{\Delta M})}$	> 1 means metric is more precise than goal. is this metric more precise than measuring the goal directly?

Real-World Use Case: Testing a New Product Recommendation Algorithm.

⇒ You're running A/B tests on a new recommendation engine for an e-commerce platform

Goal Metric

- Revenue per user (RPU) - what we ultimately care about.

Candidate Metric.

You consider two metric to evaluate the fast impact quickly:

Real Metrics

Candidate Metric	Description
CTR on Recommendations (click-Through Rate)	% of users clicking on recommended items - faster feedback, but not money
Add-to-cart Rate (ATC rate)	% of users who add a recommended item to cart -- closer to purchase intent.

Math setup

Assume from historical data

Quantity	CTR metric	ATC Metric
α (causal alignment)	0.5 (weaker)	0.9 (stronger)
σ (noise std dev)	1.2 (clicks vary a lot)	0.5 (less variance)
T = std of goal (APU)	1 (normalized scale)	1
n = users per variant	1000	1000
Power required ($1-\beta$)	80%	80%
Confidence level ($1-\alpha_0$)	95%	95%

Step-by-step Calculations

1. ☑ Alignment (correlation to Goal)

$$\rho = \frac{\alpha \cdot T}{\sqrt{\alpha^2 \cdot T^2 + \sigma^2}}$$

- CTR:

$$\rho_{CTR} = \frac{0.5}{\sqrt{0.25 + 1.44}} = \frac{0.5}{\sqrt{1.69}} \simeq 0.38$$

- ATC

$$P_{ATC} = \frac{0.9}{\sqrt{0.81 + 0.25}} = \frac{0.9}{\sqrt{0.81+0.25}} = \frac{0.9}{\sqrt{1.05}} \approx 0.87.$$

2) Sensitivity (MDE = minimum detectable good lift)

use:

$$\Delta^* = \frac{z_{1-\alpha/2} + z_{1-\beta}}{\alpha} \cdot \sqrt{\frac{2(\alpha^2 + \sigma^2)}{n}}$$

with $z_{0.975} = 1.96$ / $z_{0.8} = 0.84$

CTR:

$$\Delta^*_{CTR} = \frac{2.8}{0.5} \cdot \sqrt{\frac{2(0.25 + 1.44)}{100}} = 5.6 \cdot \sqrt{\frac{3.38}{1000}}$$

$$\approx 5.6 \cdot 0.058 \approx 0.78$$

ATC

$$\Delta^*_{ATC} = \frac{2.8}{2.9} \sqrt{\frac{2(0.81 + 0.25)}{1000}} \approx 3.11 \cdot \sqrt{2.12/1000}$$

$$= 3.11 \cdot 0.046 \approx 0.30$$

✓ ATC can detect small lifts

⚠ CTR needs a big shift (> 70%) to be statistically reliable.

Interpretation Table

Criterion	Good Range	CTR Metric	AIC Metrics
Alignment P	> 0.6	0.38 X	0.87 V
sensitivity (MDC)	< 0.4 · SD(APU)	0.78 X	0.20 V
Bias ($\alpha \approx 1$)	Close to 1	$\alpha \approx 60$ X	$\alpha \approx 0.9$ V
Noise Ratio	$\sigma^2/(\alpha^2\tau^2) < 1$	1.44/0.25 ≈ 5.8 X	0.25/0.81 ≈ 0.31 V

Conclusion

Metric	Verdict	Reason
Add-to-Cart	Accept V	Strong alignment, low noise, good sensitivity.
CTR	Reject X	Poor alignment, too noisy, weak indicator.

(5) How to Evaluate Metrics in Real World

Step	What You Do	Why.
1	Run same experiment with multiple candidate metrics	See which one behaves like the goal.
2.	Calculate correlation between each metric and your true goal	check alignment
3.	Measure how often the metric gives correct direction (positive or negative)	Check sensitivity
4.	Run "A/A" test (no treatment)	Check false positives.
5.	Slice data (device, region, traffic source)	Ensure metric is robust & fair.

Scenario: Online Store tests a New Checkout Flow.

Goal: Improve Revenue per User (RPU)
But RPU takes

Candidates Metrics

- Metric 1: Add to Cart Rate (ATC)
- Metric 2: Checkout Start Rate (CSR)

Step-by-Step Metric Evaluation (Simple Real-World)

Step	What You Do	Example	Why
1	Run same A/B test with multiple metrics	You test a new UI: Group A sees old checkout, Group B sees new one. You record RPU, ATC, CSR for both group.	See which candidate metric (ATC or CSR) behaves like RPU.
2	Correlate with RPU	You calculate Correlation(RPU, ATC) ≈ 0.82, Correlation (RPU, CSR) ≈ 0.41	✓ ATC tracks RPU more closely → better alignment.
3	Directional agreement	You run the same test 10 times in 9/tests, ~~where~~ when RPU↑, ATC↑ too. CSR only matched RPU in 5/10 tests.	✓ ATC consistently predicts the direction of good movement.
4	Run A/A test	You split users 50/50 but don't change anything. ATC difference is close to 0. CSR fluctuate randomly.	✓ ATC shows low false positives, CSR is noisy.

5	Slice the data	on mobile, AIC holds up; on desktop, CSR spikes (possibly gamed)	☑ AIC is robust across segments; CSR is not.

(6) How it Connects with other Concepts.

- Formulating Metrics: → You define the metric
- Evaluating Metrics → You test it on real data.
- Guardrails → Also must be evaluated. A bad guardrail = False alarms.
- Evolving Metrics → if a metrics get worse over time, you revise it.

(7) Final summary

Good Metric = Aligned + Sensitive.

☑ Correlation with goal > 0.6.

☑ Correct sign > 70% of time

☑ Works well across segments.

☑ Easy to explain and track.

④ Evolving Metrics.

1) What is "Evolving Metrics"?

→ Metric are not permanent. You must update them over time because your product, users or data changes.
If you don't evolve them, they'll mislead you.

1. Product Changes Break old Metrics.

Example
You used to measure success with:
→ "Add to Cart" rate.

Then, your team changes the interface
→ Adds "Wish list" button instead of "Add to Cart"

Now: people use Wishlist more, not "Add to Cart"
but they still buy later, your old metric looks bad, even though sales are good.

☑ Fix: Update the metric to count "Wish list-to-Purchase" as part of it.

2. Teams or Users "Game" the Metric

Example:
Your metric = click Through rate (CTR)
A team increases CTR by showing click bait recommendations. More people click, but fewer people actually buy. Your metric is high, but your real goal (sales) goes down.

☑ Fix:

Use smarter metric like:

→ Clicks + Dwell Time

→ Clicks that lead to a purchase.

(3). You get Better Data Over Time

Example

You used to track revenue only after checkout.

Now your system can log revenue instantly after payment, with coupon info, user ID, etc.

☑ Fix:

Build cleaner, more accurate metric like

→ Revenue per user per day

→ Revenue from paid vs free users separately.

Final Idea (Easy Analogy).

Your metric is like a compass

→ if the product is a ship, and the ocean changes — you must adjust the compass, or you'll sail the wrong way.

Math Recap: Why Metrics Drift

Metrics evolve when the parameters change in the model.

$$\Delta M = \alpha(t) \cdot \Delta G + \epsilon(t)$$

where:

- $\alpha(t)$: alignment changes over time due to UX logic, behavior shifts.
- $\varepsilon(t)$: noise changes with abuse, bugs, and external factors.

If you don't track and re-estimate α and σ regularly, your metric stops being useful.

(2) Three-step loop:

Step	What you do	Real-world use
Monitor	Track rolling correlation with true business goal	chart above shows yellow (old) drifting
Diagnose	When alignment or power slips below a threshold, inspect causes (new features, abuse, logging gap)	At experiment #100 the checkout flow changed, breaking the metric.
Re-formulate	Redefine metric: add filters, normalize per users, or create a composite (super, weighted events)	We launched the new metric (orange) with restored $\alpha \approx 0.9$ and lower σ.

3. Math recap

"Old vs. New Metric"

Old Metric

- At ~~Metric~~ first:

$$\Delta M = 0.9 \cdot \Delta G + \varepsilon_{0.7}$$

→ Metric is very aligned (90%), but has some noise (0.7)

- later (after product changes)

$$\Delta M = 0.4 \cdot \Delta G + \varepsilon_{0.7}$$

→ Now it barely tracks the goal (only 40%), but same noise.
→ It's not trustworthy anymore.

New Metric

- After updating

$$\Delta M = 0.9 \cdot \Delta G + \varepsilon_{0.5}$$

→ Back to strong alignment (0.9)
→ Add new and less noise (0.5) = more stable, more accurate.

Real-World Meaning

Metric	Tracks the real world?	Is it noisy?	Trust it in Experiments?
old (before)	☑ Yes (0.9 x real goal)	x somewhat	✓ Yes
old (after	x Poor (0.4 x real goal)	x Somewhat	x No (confusing).
New	✓ Yes (0.9 x real goal)	✓ less noise	✓ Yes (ideal)

Final Part: Power ≈ 1.0

we calculate the "power" (how well this metric detects real change, like revenue increase)

→ A power of 1.0 = 100% chance of detecting a 0.3 increases in goal if it's real.

→ So even both metrics can detect changes, but:

- Old metric is misleading (wrong direction or inconsistent)
- New metric is misleading is both powerful and aligned (best choice).

Summary: In very simple words.

- Your metric is a mirror to your real goal.
- If it stops reflecting the real thing, or becomes foggy (noisy) - you must change it.
- Math helps us measure how good or bad that mirror is.

4. Analogy

Weather station

- old thermometer drift after years in sun → recalibrate or replace.
- Same for metrics: recalibrate when "heat" (goal) reading no longer matches reality.

5. How it interlocks

Topic	Relations
Evaluating	finds drift - triggers evolution
Gameability	New metric removes exploitable loopholes.
Guardrails	Guardrails threshold may tighten as metric improves.

(5) SLOS BAR - Guardrails Metric

1. What is Guardrail Metric?

→ A guardrail metric is like a safety rule in an experiment:

- ↳ It's not your goal
- ↳ You're not trying to make it better - you just want to make sure it doesn't get worse
- ↳ If it gets worse beyond a threshold → experiment fails, even if the main goal improves.

Real-world Analogy: Fast Car on a Highway

Element	Meaning
Speed	Your goal metric (e.g. revenue ↑)
Guardrail	Highway rail (e.g. latency ≤ 1.5 sec)
Crash	User pain (slow, broken, frustrating)
Conclusion	Speed is great - but crash = failure.

So if your new checkout increases revenue but causes pages to load in 4 seconds, the "car" (your site) just crashed.

You never want to win the race by wrecking the user experience.

How Guardrail Metrics Are Used in Practice

During an A/B Test

1. Your goal metric : Conversion rate ↑

2. Your guardrail metric : Error rate ≤ 0.5% /
Page load ≤ 2.5s

3. If :
   - Conversion rate increases ✓
   - But page speed becomes 3.2s - ✗

Result : The Feature Fails despite hitting the goal.

Common Guardrail Metrics by Domain

Domain	Guardrail Metric (examples)
E-commerce	Page load time, checkout failure rate.
Mobile apps	App crash rate, battery usage spike.
Finance	loan default rate, fraud detection misses.
Video platform	Playback failure rate, buffering time
Messaging apps	Delivery latency / push opt-out rate

Guardrails are Not Optional.

Guardrails protect long-term trust. You can't just optimize for clicks, sign-ups, or revenue while hurting reliability, performance, or fairness

They create a fail-fast system:
If a new feature breaks something, you catch it immediately and stop rollout.

Memory Hook: "Goal Lifts, Guardrails Hold"

A helpful phrases:

> "Your goal metric lifts, but your guardrail metrics hold"

This is what a successful experiment looks like.

2. Good Guardrail Rules.

1. Rule 1: Bounded
> "The metric must have a clear threshold or limit"

Example:
- 95th percentile page-load time ≤ 2.5 seconds
- Crash rate ≤ 1%

This makes the rule binary - clear Pass or Fail.

Why it Matters:
without a boundary, you end up debating feelings, not facts.

x	"This feels a little slow"
✓	"Nope, latency hit 2.8 sec. Guardrail failed"

2. Rule 2: Fast to Measure.

> "The metric should show problems quickly"

Meaning:
You can detect a failure early in the experiment, not wait weeks.

Example:
- App crash rate can be seen within minutes of release.
- Page latency is visible from first 100 users.

Why it Matters:
Fast guardrails = faster rollback = less user pain.

Imagine:
- New feature causing 40% crashes
- Do you want to find that now, or next week?

3. Rule 3 : Hard to Game

> "The metric should reflect reality — not what teams make it look like"

Meaning

Guardrails should be immune to manipulation.

Example

- Don't rely on client-reported speed (teams might cheat it)
- Instead use server-side latency logs.

> ✗ Team might delay metric reporting to hide problems
> ✓ Server logs don't lie — They show the truth.

Why it Matters

If a metric is gameable, it loses its value as a true safety check.

4 Rule 4 : Critical

> "The metric must protect user experience, safety, or trust"

Meaning:

If this breaks, it's a big deal — not just a cosmetic issue.

Example

- Data Loss
- Uninstalled logouts
- Users clicking "Unsubscribe" or uninstalling the app

Why it Matters

Guardrails should be user-first — they protect the core experience.

> You don't want revenue to go up if trust, reliability, or privacy go down.

3. Math concept

Let G = goal metric uplift (want ↑)
Let R = guardrail metric change (want ≤ 0)

Decision rule:

ship if: $\begin{cases} G \geq +MDE & \text{(goal wins)} \\ R \leq 0 & \text{(no harm)} \end{cases}$

You run two hypothesis tests:

1. $H_0^G : \Delta G = 0$ - need significant increase.

2. $H_0^R : \Delta R > 0$ - need fail-safe (non-increase)

if either fails - roll back.

Example: Scenario

You're testing a new homepage design

- Goal: increase conversion rate (CVR)
- Guardrail: keep page load time (PLT) under control.
- Constraint: 95th percentile PLT ≤ 2.5 seconds

A/B Experiment Data

Group	Sample size	Conversion	95th %ile PLT (sec)
Control	10,000	1,100	2.30 ✓
Treatment	10,000	1,250	2.85 ✗

Step 1: Goal Metric = Conversion Rate.

$$CVR = \frac{\text{Conversions}}{\text{Users}}$$

- Control CVR = 1100 / 10000 = 11%.
- Treatment CVR = 1250 / 10000 = 12.5%.

$$\Delta CVR = 12.5\% - 11\% = +1.5\%$$ (Goal Improved ✓)

Step 2: Guardrails Metric = Page Load Time

We said:

| Guardrail = 95th percentile page load ≤ 2.5s.

- Control: 2.30s ✓
- Treatment: 2.85s ✗

Guardrail failed.

Step 3: Final Decision

Even though the treatment improves conversion, it violates the guardrail.

| ① Result: Do not ship the new homepage

This prevents you from rolling out something that may:

- Increase bounce rates
- Harm SEO
- Reduce user trust on slower connections.

Optional Math (Statistical check for Guardrail violation)

Two-Sample Z-test (Unpaired, Two-Tailed)

Why?

- You're comparing the means of two independent groups.
  - Control group (old version)
  - Treatment group (new version)

$$Z = \frac{\bar{X}_1 - \bar{X}_2}{\sqrt{\frac{\sigma_1^2}{n_1} + \frac{\sigma_2^2}{n_2}}}$$

Where:

- $\bar{X}_1, \bar{X}_2$: means of the two group
- σ_1, σ_2 : standard deviations.

- n_1, n_2 : sample size (30 days, in our case)
- Assumes independent observations.

Given data

- Mean of control group : 2.30s
- Mean of treatment group: 2.85s
- standard deviation for both (daily samples) : 0.05s
- Number of days sampled for each : 30 days.

z - Formula

The formula is

$$z = \frac{\text{difference in means}}{\text{standard error (SE)}}.$$

$$SE = \sqrt{\frac{SD^2}{n} + \frac{SD^2}{n}} = \sqrt{\frac{0.05^2}{30} + \frac{0.05^2}{30}}$$

Plug in the values

(a) Compute SE

$$SE = \sqrt{\frac{0.0025}{30} + \frac{0.0025}{30}} = \sqrt{0.00\,00833 + 0.00\,00833}$$

$$= \sqrt{0.0001667} = 0.0129$$

(b) Compute z-score

$$z = \frac{2.85 - 2.30}{0.0129} = \frac{0.55}{0.0129} \approx 42.6$$

Interpreting z-scores

z-score	Interpretations
≈ 0	No difference from expected (null)
±1 to 2	Slightly unusual, but could easily be random
> 2	Statistically significant ($p < 0.05$).
> 3	Highly significant ($p < 0.01$ or less).
> 5 or 10	Extremely unlikely by chance.

Goal Type	Desired z-score.
Goal Metric (e.g. conversion rate ↑)	want $z > +2$ (prove improvement).
Guardrail Metric (e.g. latency stays flat)	want $z \approx 0$ or < 2 (no meaningful change)

Why is a z-score of 42.6 such a big deal? what does it really mean?

→ p-value tells you!

- if $z = 2 \rightarrow p \approx 0.045 \rightarrow 4.5\%$ chance this happened randomly.
- if $z = 42.6 \rightarrow 0.000\,00\ldots001 \rightarrow$ no way this happened randomly.

| The result is so extreme it's 100% likely to be real – not Chance.

(5) Real-world Analogies

- Hospital: Reduce surgery throughput (goal) but guardrail = mortality rate must not rise
- Banking app: Increase loan approvals (goal) but guardrail = default rate cannot exceed 1%.
- Video Service: Boost watch time (goal) but guardrail = playback error rate below 0.1%.

(6) Overlap with Other Topics

Topics	Interaction
Gameability	Guardrails reduce incentive to game driver metrics
Balancing Metrics	Guardrail metrics must themselves be tested for stability.
Evolving Metrics	Guardrail thresholds tighten or adapt as product and infra improve.

Take - away

Guardrail metrics act like a red light - they stop release that might increase business KPIs at the cost of breaking user experience or trust. Never ship when a guardrail turn red.

(6) SIDEBAR : Gamebility

(1) What is "Gamebility" in Metrics?

Gamebility means A metric can be manipulated. to make an experiment look good, even what it isn't truly good for users or the product.

In simple terms

> "You can trick the metric without improving the actual experience".

(2) Real world Analogy : School Grading

Imagine your school switches from exams to only grading homework completion.

Students quickly learn:

> "If I just copy answers and submit everything, I get an A - even if I learn nothing".

The metric (homework done) is now gameable — it can be optimized without achieving the true goal (learning).

③ In A/B Testing Context.

x Gameable Metric Example

- Click-through Rate (CTR) as your success metric.
  - → You make the button super bright and spammy-looking
  - → CTR increases — metric says "success".
  - → But users don't actually convert or may feel tricked.

Business Impact = bad
Metric impact = looks good
→ Metric was gameable.

④ Hard-to-Game Metric (Good)

Use downstream, meaningful metrics.
- ↳ Instead of "Did user click?"
- ↳ Use "Did user complete purchase within 7 days?"

These are harder to manipulate — because you can't fake long-term value.

Connections to other Topics.

Related Topics	Relationship.
Guardrail Metrics	These are designed to be hard to game, and protect from bad experiments.
Formulating Metrics	Emphasizes building robust, non-gameable metrics.
Evaluating Metrics	Test if a metric is sensitive and valid - not gameable.

Examples with Simple Math

Let's say we use:

- CTR (click-through rate) = clicks / impressions
  → Goal: Increase CTR
  → Team adds fake "claim Reward" button.

Variant	Impressions	clicks	CTR
Control	1000	100	10%
Treatment	1000	300	30%

✓ Looks good on paper
✗ But downstream: only 5% of these clicks convert → false signal.

Final Takeaways

principle	Good Metrics should be.
Aligned	with long-term product or user value.
Robust	Not manipulated by UX tricks or spam.
Hard to Game	Can't be hacked or boosted superficially
Downstream	Closer to real impact (e.g. purchase, retention)